Deep Learning in Computer Vision· Lecture 2 of 9

Training Neural Networks

Loss functions, backpropagation and optimizers

Loss Functions: Measuring Error

- The **loss** is one number saying how wrong the predictions $\hat{y}$ are

- *Important Applications:*
  - **Regression** is a supervised learning task where the model is trained to predict a **continuous numerical value** (e.g., predicting house price, temperature, or age).
    - Typically consists of a single neuron (or multiple neurons for multi-output regression)
  - **Classification** predicts **class probabilities** (e.g., predicting whether an image is a cat, dog, or bird).

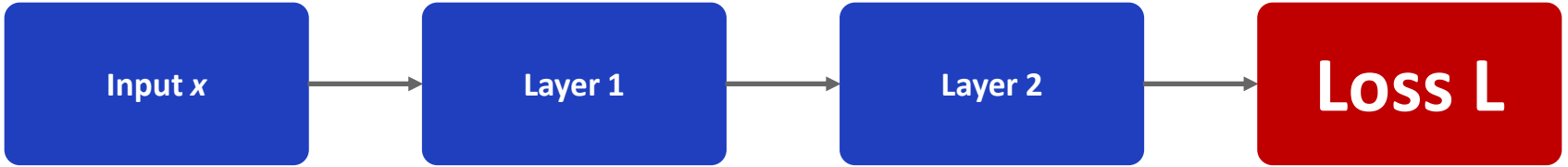
Loss Functions: Measuring Error

- The **loss** is one number saying how wrong the predictions $\hat{y}$ are
- *Important Notations:*
  - N: The total number of samples or data points in the dataset (or batch).
  - y: The **true label** or actual target value (ground truth).
    - In regression, y is a continuous target number.
    - In classification, y is typically a **one-hot encoded** vector indicating the true class (e.g., $[0,1,0]$) outputted by functions like linear/identity or ReLU.
  - $\hat{y}$ **("y-hat")**: The **predicted value** or output generated by the model.
    - In regression, $\hat{y}$ is the predicted continuous number.
    - In classification, $\hat{y}$ is the predicted probability for a class outputted by functions like Softmax or or Sigmoid (binary) (e.g., $\hat{y}$ = [0.05, 0.85, 0.10]).

Loss Functions: Measuring Error

- The **loss** is one number saying how wrong the predictions are
- Regression: **Mean Square Error (MSE)** = $\left(\frac{1}{N}\right) \Sigma\, (y - \hat{y})^2$
  - MSE calculates the average squared difference between true targets y and predicted outputs $\hat{y}$ across all N samples
- Classification: **cross-entropy** = $-\Sigma\, y \cdot \log(\hat{y})$
  - Measures the difference between two probability distributions (the ground truth $y$and predicted probabilities $\hat{y}$). Taking the negative log penalizes predictions strongly when the model assigns a low probability to the correct class
- Softmax turns raw scores into probabilities that sum to 1
- **Training = find weights that minimize the loss**

Backpropagation



forward →

- We feed input x into Layer 1, get intermediate hidden output h, pass it to Layer 2, get output out, and calculate **Loss** L at the very end.
- **Goal of Training:** Find the weights (like w_1) that make **Loss** L **as small as possible**.

← gradients flow backward

Recall: In plain English, $\frac{\partial A}{\partial B}$ answers the question:
"If I make a tiny tweak to B, by how much will A change?"

Chain rule: $$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial h} \cdot \frac{\partial h}{\partial w_1}$$

Each weight gets its own gradient: $\frac{\partial L}{\partial w_1}$ **how much the loss L changes if that weight w_i changes.**

w_1 affects h
h affects out
out affects L

Optimizers…..

- Optimizers, combined with their cousin the loss function, are the key pieces that enable machine learning to work for *your* data.
- During the training process, we tweak and change the parameters (weights) of our model to try and minimize that loss function, and make our predictions as correct and optimized as possible
- But how exactly do you do that? How do you change the parameters of your model, by how much, and when?
- **This is where optimizers come in.** They tie together the loss function and model parameters by updating the model in response to the output of the loss function to reach the global minima

Optimizer Intuition: The Hiker

- Hiker in fog: cannot see the valley but feels **uphill vs downhill**
- Keep stepping downhill → reach the valley
- The optimizer does this on the **loss surface**
- Goal: reach a (good) minimum of the loss

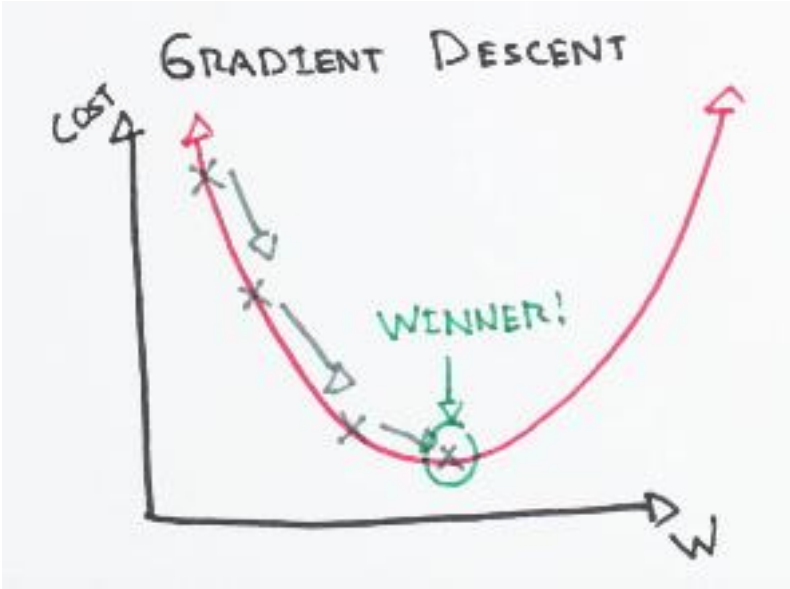


Optimizers: Examples

- The Examples are:
  - Gradient Descent
  - Stochastic Gradient Descent
  - Adam
  - RMSProp

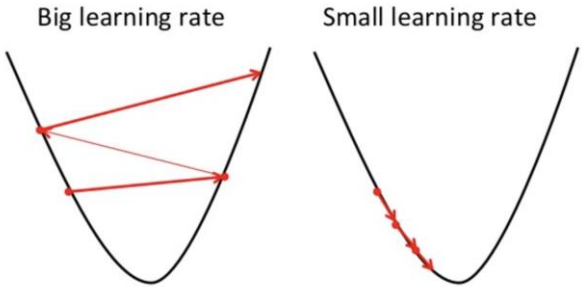
Gradient Descent

- 1. Compute the **gradient** of the loss for every weight
- 2. Step **against** it: $\mathbf{w_{new}} \leftarrow w_{old} - \eta \times \frac{\partial L}{\partial w}$
  - η is the **learning rate** (step size)
- 3. Repeat until the loss stops decreasing



Note: The **Learning Rate (η or α)** controls the size of the step a neural network takes in the direction opposite to the gradient during weight updates:

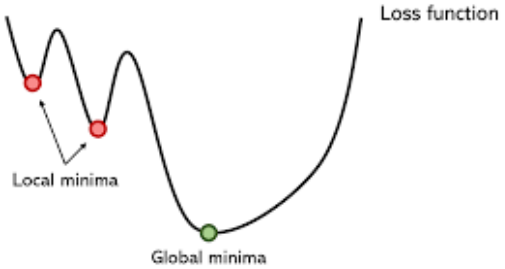
Learning Rate



- **Too large**: overshoots, loss bounces or diverges
    - Changing our weights too fast by adding or subtracting too much (i.e. taking steps that are too large) can hinder our ability to minimize the loss function.
- **Too small**: very slow, may stall
    - Typical initial learning start: 10^{-3} (Adam), 10^{-2} (Stochastic Gradient Descent (SGD)) to build momentum
- Learning rate is often **decayed** during training
    - Start training with a relatively high learning rate and **gradually reducing (shrinking) it over time as training progresses**.

Challenges for Gradient Descent

- You might experience during optimization to get stuck on local minima
  - When dealing with high dimensional datasets (lots of variables), it's possible you'll find an area where it *seems* like you've reached the lowest possible value for your loss function, but it's really just a ***local* minimum**
- To avoid getting stuck in local minima, we make sure we use the proper learning rate



- **Important Terminology**
  - **Local minima**: flat valleys that are not the best
  - **Saddle points**: Points where the gradient is zero ($\nabla L = 0$),

    but the surface goes uphill in some directions and downhill in others (shaped like a horse saddle).
  - **Ill-conditioned** surfaces: zig-zag progress
  - **Plateaus:** Large, flat regions on the loss surface where gradients are extremely tiny ($\nabla L \approx 0$)
- Note: **In deep nets most stalls are saddles/plateaus**, not poor minima

Stochastic Gradient Descent (SGD)

- Instead of calculating the gradients for *all of your training examples* on every pass of gradient descent
  - it's sometimes more efficient **to only use a subset of the training examples each time**

- Stochastic Gradient Descent is an implementation that either uses batches of examples at a time or random examples on each pass.

Batch, Mini-batch and Stochastic GD

Variant	Samples per update	Pros	Cons
Batch GD	All N	Stable, exact gradient	Slow, memory heavy
SGD	1	Fast, noisy (helps escape)	Very noisy
Mini-batch	32–256	Fast + stable, GPU-friendly	Needs batch size choice

In practice "SGD" almost always means mini-batch SGD. One pass over the data = one epoch.

Momentum and RMSProp

- This slide introduces **Momentum** and **RMSProp**, two distinct mechanisms designed to fix limitations in **GD** (e.g. **Saddle Points & Plateaus)**.
- **Momentum**: simulates a heavy ball rolling down a hill. Instead of relying purely on the current step's gradient, it accumulates velocity from previous updates; smooths zig-zags and speeds up. It is a 1st moment, tracking gradient direction/velocity
  - **The Update Equations:**

$$v \leftarrow \beta v + \nabla L$$

$$w \leftarrow w - \eta v$$

    - v **(Velocity):** A running accumulation of past gradients.
    - β **(Friction / Decay Factor, typically** ≈ 0.9**):** Determines how much velocity from past steps is retained.
    - ∇L **(Current Gradient):** The acceleration added in the current step.
    - η **(Learning Rate):** The step multiplier.

RMSProp: (Root Mean Square Propagation)

- introduces an **adaptive, per-weight learning rate** based on recent gradient magnitudes. It is a 2nd moment, tracking gradient magnitude.
- tracks an exponentially decaying average of **squared gradients** (s):

$$\boldsymbol{s} \leftarrow \gamma s + (1-\gamma)(\boldsymbol{\nabla L})^2$$

$$w \leftarrow w - \frac{\eta}{\sqrt{\boldsymbol{s}+\epsilon}} \odot \boldsymbol{\nabla L}$$

ϵ: **Smoothing Term,**
γ: **Decay Rate,**
$\odot$: **Element-wise multiplication operator**

  - If a parameter has small, gentle gradients, $\boldsymbol{s}$ remains small. Dividing by $\sqrt{\boldsymbol{s}}$ boosts the effective step size, speeding up progress along flat plateaus.
  - If a parameter has consistently large gradients, $\boldsymbol{s}$ grows large. Dividing by $\sqrt{\boldsymbol{s}}$ scales down the effective learning rate, dampening wild oscillations.
- Steep directions get smaller steps, flat ones larger

Adam

- Adam stands for **ada**ptive **m**oment estimation
- Adam is another way of using past gradients to calculate current gradients
- Adam also utilizes the concept of momentum by adding fractions of previous gradients to the current one.

- **Adam** uses both **Momentum** (1st moment, tracking gradient direction/velocity) and **RMSProp** (2nd moment, tracking gradient magnitude) simultaneously.

- This optimizer has become pretty widespread, and is practically accepted for use in training neural nets

Adam: Adaptive Moment Estimation

Momentum part

Average of past gradients (1st moment).

RMSProp part

Average of squared gradients (2nd moment).

Adam = Momentum + RMSProp (+ bias correction)

Widely used default optimizer: Learning Rate = 0.001 works well in many tasks and needs little tuning.

Choosing an Optimizer

Optimizer	Idea	Typical use
SGD	Plain mini-batch step	Baseline; needs tuned η
SGD + momentum	Smoothed direction	CNNs (ResNet, VGG) with lr 0.01–0.1
RMSProp	Per-weight scaling	RNNs, some RL
Adam	Momentum + scaling	Fast default for most new projects

Rule of thumb: start with Adam; try SGD + momentum when you need the last bit of accuracy.

Recap

- Loss measures error; **cross-entropy** for classification
- **Backprop** computes gradients via the chain rule
- **Optimizers** use gradients to update weights: GD → SGD → Momentum → Adam
- Learning rate is the most important hyper-parameter
- Next: **overfitting, imbalance, vanishing gradients, metrics**